



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA E
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 10

NOMBRE COMPLETO: Vargas Luna José Ángel

N° de Cuenta: 318252333

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 07

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 10-05-2026

CALIFICACIÓN: _____

```

//////////////////////////////////KEYFRAMES//////////////////////////////////

bool animacion = false;

//NEW// Keyframes
float posXavion = 2.0, posYavion = 2.0, posZavion = 0;
float movAvion_x = 0.0f, movAvion_y = 0.0f;
float giroAvion = 0;

// Variables para animación de alas
float giroAla = 0.0f; // ángulo actual del ala
float giroAlaDir = 1.0f; // dirección del aleteo (1 o -1)
float giroAlaMax = 25.0f; // amplitud máxima del aleteo en grados
float giroAlaVel = 3.0f; // velocidad de aleteo

#define MAX_FRAMES 100
int i_max_steps = 120;
int i_curr_steps = 0;
typedef struct _frame
{
    //Variables para GUARDAR Key Frames
    float movAvion_x; //Variable para PosicionX
    float movAvion_y; //Variable para PosicionY
    float movAvion_xInc; //Variable para IncrementoX
    float movAvion_yInc; //Variable para IncrementoY
    float giroAvion;
    float giroAvionInc;
}FRAME;

FRAME KeyFrame[MAX_FRAMES];
int FrameIndex = 19;
bool play = false;
int playIndex = 0;

```

En este apartado declaramos los keyframes, donde declaramos las variables de movimiento del avión en X & Y, incremento en las mismas variables y el giro.

```

void saveFrame(void) //tecla L
{
    printf("Frameindex %d\n", FrameIndex);

    KeyFrame[FrameIndex].movAvion_x = movAvion_x;
    KeyFrame[FrameIndex].movAvion_y = movAvion_y;
    KeyFrame[FrameIndex].giroAvion;
    //no volatil, agregar una forma de escribir a un archivo para guardar los frames
    FrameIndex++;
}

void resetElements(void) //Tecla 0
{
    movAvion_x = KeyFrame[0].movAvion_x;
    movAvion_y = KeyFrame[0].movAvion_y;
    giroAvion = KeyFrame[0].giroAvion;
}

void interpolation(void)
{
    KeyFrame[playIndex].movAvion_xInc = (KeyFrame[playIndex + 1].movAvion_x - KeyFrame[playIndex].movAvion_x) / i_max_steps;
    KeyFrame[playIndex].movAvion_yInc = (KeyFrame[playIndex + 1].movAvion_y - KeyFrame[playIndex].movAvion_y) / i_max_steps;
    KeyFrame[playIndex].giroAvionInc = (KeyFrame[playIndex + 1].giroAvion - KeyFrame[playIndex].giroAvion) / i_max_steps;
}

```

En este apartado declaramos los Keyframes, donde en saveFrame() guardamos la posición actual de la nave e incrementamos el índice.

En resetElements() regresamos la nave a su posición inicial

Con interpolation() calcula los incrementos en cada paso entre el frame actual y el siguiente.

```

void animate(void)
{
    // Animación continua del ala (aleteo) - siempre activa
    giroAla += giroAlaVel * giroAlaDir;
    if (giroAla >= giroAlaMax || giroAla <= -giroAlaMax)
        giroAlaDir *= -1.0f; // invertir dirección del aleteo

    //Movimiento del objeto // barra espaciadora
    if (play)
    {
        if (i_curr_steps >= i_max_steps) //end of animation between frames?
        {
            playIndex++;
            printf("playindex : %d\n", playIndex);
            if (playIndex > FrameIndex - 2) //end of total animation?
            {
                printf("Frame index= %d\n", FrameIndex);
                printf("termina anim\n");
                playIndex = 0;
                play = false;
            }
            else //Next frame interpolations
            {
                //printf("entro aquí\n");
                i_curr_steps = 0; //Reset counter
                //Interpolation
                interpolation();
            }
        }
        else
        {
            //printf("se quedó aquí\n");
            //printf("max steps: %f", i_max_steps);
            //Draw animation
            movAvion_x += KeyFrame[playIndex].movAvion_xInc;
            movAvion_y += KeyFrame[playIndex].movAvion_yInc;
            giroAvion += KeyFrame[playIndex].giroAvionInc;
            i_curr_steps++;
        }
    }
}

```

Y en animate() aplica los incrementos de cada frame y también ejecuta el aleteo continuo del ala.

```

//Keyframes
glm::vec3 posblackhawk = glm::vec3(2.0f, 0.0f, 0.0f);

// [0] Origen - punto de partida
KeyFrame[0].movAvion_x = 0.0f;
KeyFrame[0].movAvion_y = 0.0f;
KeyFrame[0].giroAvion = 0.0f;

// --- ZIGZAG IDA ---
// [1] Pico 1 - sube
KeyFrame[1].movAvion_x = 2.0f;
KeyFrame[1].movAvion_y = 3.0f;
KeyFrame[1].giroAvion = 0.0f;

// [2] Valle 1 - baja
KeyFrame[2].movAvion_x = 4.0f;
KeyFrame[2].movAvion_y = -1.0f;
KeyFrame[2].giroAvion = 0.0f;

// [3] Pico 2 - sube
KeyFrame[3].movAvion_x = 6.0f;
KeyFrame[3].movAvion_y = 3.0f;
KeyFrame[3].giroAvion = 0.0f;

// [4] Valle 2 - baja (antes de entrar al loop)
KeyFrame[4].movAvion_x = 8.0f;
KeyFrame[4].movAvion_y = -1.0f;
KeyFrame[4].giroAvion = 0.0f;

// --- LOOP ---
// [5] Entrada al loop / punto medio
KeyFrame[5].movAvion_x = 9.0f;
KeyFrame[5].movAvion_y = 1.0f;
KeyFrame[5].giroAvion = 0.0f;

// [6] Loop - tope superior
KeyFrame[6].movAvion_x = 10.0f;
KeyFrame[6].movAvion_y = 4.0f;
KeyFrame[6].giroAvion = 0.0f;

// [7] Loop - lado derecho
KeyFrame[7].movAvion_x = 11.0f;
KeyFrame[7].movAvion_y = 1.0f;
KeyFrame[7].giroAvion = 0.0f;

// [8] Loop - fondo inferior
KeyFrame[8].movAvion_x = 10.0f;
KeyFrame[8].movAvion_y = -2.0f;
KeyFrame[8].giroAvion = 0.0f;

// [9] Loop - cierre, regresa al punto de entrada del loop
KeyFrame[9].movAvion_x = 9.0f;
KeyFrame[9].movAvion_y = 1.0f;
KeyFrame[9].giroAvion = 0.0f;

// --- GIRO EN SITIO al final del recorrido ---
// [10] Misma posición, inicia giro
KeyFrame[10].movAvion_x = 9.0f;
KeyFrame[10].movAvion_y = 1.0f;
KeyFrame[10].giroAvion = 0.0f;

// [11] Misma posición, giro completado 180°
KeyFrame[11].movAvion_x = 9.0f;
KeyFrame[11].movAvion_y = 1.0f;
KeyFrame[11].giroAvion = 180.0f;

// --- ZIGZAG REGRESO (espejo exacto de la ida) ---
// [12] Valle 2 - baja (regreso)
KeyFrame[12].movAvion_x = 8.0f;
KeyFrame[12].movAvion_y = -1.0f;
KeyFrame[12].giroAvion = 180.0f;

// [13] Pico 2 - sube (regreso)
KeyFrame[13].movAvion_x = 6.0f;
KeyFrame[13].movAvion_y = 3.0f;
KeyFrame[13].giroAvion = 180.0f;

// [14] Valle 1 - baja (regreso)
KeyFrame[14].movAvion_x = 4.0f;
KeyFrame[14].movAvion_y = -1.0f;
KeyFrame[14].giroAvion = 180.0f;

```

```

// [15] Pico 1 - sube (regreso)
KeyFrame[15].movAvion_x = 2.0f;
KeyFrame[15].movAvion_y = 3.0f;
KeyFrame[15].giroAvion = 180.0f;

// [16] Origen - nave llega al punto inicial apuntando hacia la ida
KeyFrame[16].movAvion_x = 0.0f;
KeyFrame[16].movAvion_y = 0.0f;
KeyFrame[16].giroAvion = 180.0f;

// --- GIRO EN SITIO en el origen ---
// [17] Misma posición, inicia giro de vuelta
KeyFrame[17].movAvion_x = 0.0f;
KeyFrame[17].movAvion_y = 0.0f;
KeyFrame[17].giroAvion = 180.0f;

// [18] Misma posición, giro completado 360° (=0°), lista para repetir
KeyFrame[18].movAvion_x = 0.0f;
KeyFrame[18].movAvion_y = 0.0f;
KeyFrame[18].giroAvion = 360.0f;

printf("\nTeclas para uso de Keyframes:\n1.-Presionar S para reproducir animacion.\n2.-Presionar R para volver a habilitar reproduccion de la animacion\n");
printf("3.-Presiona G para guardar frame\n4.-Presiona P para habilitar guardar nuevo frame\n5.-Presiona M para mover en X\n6.-Presiona H para habilitar mover en X\n");

```

En este apartado empezamos a llamar a los keyframes, donde la nave hace el recorrido entre los valles y crestas, además de iniciar el loop de la vuelta, el giro de 180° y su regreso.

```

void inputKeyframes(bool* keys)
{
    if (keys[GLFW_KEY_I])
    {
        if (reproduciranimacion < 1)
        {
            if (play == false && (FrameIndex > 1))
            {
                resetElements();
                //First Interpolation
                interpolation();
                play = true;
                playIndex = 0;
                i_curr_steps = 0;
                reproduciranimacion++;
                printf("presiona R para habilitar reproducir de nuevo la animacion\n");
                habilitaranimacion = 0;
            }
            else
            {
                play = false;
            }
        }
    }
    if (keys[GLFW_KEY_R])
    {
        if (habilitaranimacion < 1)
        {
            reproduciranimacion = 0;
            habilitaranimacion++;
            printf("Ya puedes reproducir de nuevo la animacion con la tecla S\n");
        }
    }
}

```

```

if (keys[GLFW_KEY_G])
{
    if (guardoFrame < 1)
    {
        saveFrame();
        printf("presiona P para habilitar guardar otro frame\n");
        guardoFrame++;
        reinicioFrame = 0;
    }
}

if (keys[GLFW_KEY_P])
{
    if (reinicioFrame < 1)
    {
        guardoFrame = 0;
        reinicioFrame++;
        printf("Ya puedes guardar otro frame presionando la tecla G\n");
    }
}

if (keys[GLFW_KEY_M])
{
    if (ciclo < 1)
    {
        movAvion_x += 1.0f;
        printf("movAvion_x es: %f\n", movAvion_x);
        ciclo++;
        ciclo2 = 0;
        printf("Presiona la tecla H para poder habilitar la variable\n");
    }
}

if (keys[GLFW_KEY_H])
{
    if (ciclo2 < 1)
    {
        ciclo = 0;
        ciclo2++;
        printf("Ya puedes modificar tu variable presionando la tecla M\n");
    }
}

```

En este apartado mandamos a llamar los teclados con los que se controlan los frames.

- I – Reproduce la animación
- R – Reiniciamos la animación del aleteo y recorrido de la nave.
- G – Guarda el frame actual
- P – Reinicia el guardado de un nuevo frame
- M – Incrementa el movimiento del avión para mover la nave en X.
- H – Reinicia el movimiento del ciclo en X para poder presionar la M.

Conclusiones

La práctica pone interesante el concepto de animación, considerando ahora los frames de movimientos, como funciona realmente en la animación el concepto de frames, entendiendo los recuadros y como se almacenan, además de lograr modificar sus movimientos, logrando que estos incrementen.

Referencias

Khronos Group. (s.f.). *Keyframe animation*. OpenGL Wiki.
https://www.khronos.org/opengl/wiki/Keyframe_Animation

Megabyte Softworks. (s.f.). *Tutorial 24: Animation pt. 1 – Keyframe MD2* [OpenGL 3].
<https://www.mbsoftworks.sk/tutorials/opengl3/24-animation-pt1-keyframe-md2/>

de Vries, J. (2020). *Skeletal animation*. LearnOpenGL. <https://learnopengl.com/Guest-Articles/2020/Skeletal-Animation>